

System Architecture, SOLID Principles & GoF Patterns - Detailed Notes

DETAILED COMPREHENSIVE NOTES

Subject: Software Engineering | Chapter: Chapter 2: System Architecture, SOLID Principles & Design Patterns | Academic Year: 2026-2027

1. The SOLID Principles of Object-Oriented Design

Single Responsibility Principle (SRP): A class should have one, and only one, reason to change.

Open/Closed Principle (OCP): Software entities should be open for extension, but closed for modification.

Liskov Substitution Principle (LSP): Subtypes must be substitutable for their base types without altering program correctness.

Interface Segregation Principle (ISP): Clients should not be forced to depend upon interfaces they do not use (prefer small, role-specific interfaces).

Dependency Inversion Principle (DIP): High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces).

2. Gang of Four (GoF) Design Patterns

Creational Patterns: Deal with object creation mechanisms.

Singleton: Ensures a class has only one instance and provides a global point of access.

Factory Method: Defines an interface for creating an object, deferring instantiation to subclasses.

Builder: Separates the construction of a complex object from its representation.

Structural Patterns: Deal with object composition.

Adapter: Converts the interface of a class into another interface clients expect.

Decorator: Dynamically attaches additional responsibilities to an object.

Behavioral Patterns: Deal with algorithms and communication between objects.

Observer: Defines a one-to-many dependency so that when one object changes state, all dependents are notified.

Strategy: Defines a family of algorithms, encapsulates each, and makes them interchangeable at runtime.

3. Architectural Styles & Modularity

Monolithic Architecture: Unified single codebase deploying all application tiers together. Simple initial development, but challenges in scaling independent modules.

Microservices Architecture: Collection of small, autonomous, independently deployable services communicating over lightweight protocols (REST, gRPC, message queues). Promotes polyglot tech stacks and fault isolation.